

TRƯỜNG ĐẠI HỌC HÙNG VƯƠNG TP. HỒ CHÍ MINH
KHÓA KỸ THUẬT CÔNG NGHỆ

BÁO CÁO THỰC HÀNH

LAB_1: Phân Tích Hệ Thống Nhận Dạng Vân Tay

Học phần	04211 Bảo mật sinh trắc, lớp 2610421101, học kỳ 1 năm học 2026-2027
Giảng viên	TS. Nguyễn An Khương
Họ và tên sinh viên	Nguyễn Huy Đạt
Mã số sinh viên	2305CT2334
Hình thức	Cá nhân
Ngày nộp	27/09/2026

1. Tóm tắt kết quả

Bài thực hành đã triển khai và đánh giá hoàn chỉnh hệ thống nhận dạng vân tay tự động (AFIS) trên tập dữ liệu FVC2002 DB4_B. Hệ thống bộc lộ hiện tượng bão hòa đặc trưng khi toàn bộ ảnh đều chạm trần 150 minutiae do khâu tiền xử lý tạo ra nhiều điểm giả. Khi tiến hành kiểm thử theo giao thức 30 lượt cùng ngón và 30 lượt khác ngón với ngưỡng quyết định cố định 0.35, cấu hình mặc định C0 (block_size = 11) đạt FRR là 93.33% và FAR là 3.33%. Khi thay đổi kích thước khối nhị phân sang C1 (block_size = 5) hoặc C2 (block_size = 21), hệ thống rơi vào trạng thái suy thoái với FRR chạm ngưỡng 100.00% và FAR giảm về 0.00%. Tỷ lệ lỗi dạng vân (Mismatch) được ghi nhận bằng 0 trên mọi cấu hình.

2. Mức độ hoàn thành

Bước hoặc yêu cầu trong đề	Trạng thái	Minh chứng tại mục
Bước 1: Khởi tạo Không Gian Làm Việc	Hoàn thành	4.1, 3
Bước 2: Khám Phá Quy Trình Qua Giao Diện Dòng Lệnh (CLI)	Hoàn thành	4.2
Bước 3: Tích Hợp API Vào Kịch Bản Tự Động	Hoàn thành	4.3, 5
Bước 4: Bài Tập Phân Tích Lỗi	Hoàn thành	5, 6

3. Môi trường thực hiện và khả năng tái lập

Thông tin	Giá trị
Hệ điều hành, CPU, RAM	Windows 11, Intel(R) Core(TM) i5-1135G7(DELL), 16 GB
Phiên bản Python	Python 3.13.9
Thư viện chính và phiên bản	numpy==2.5.3, opencv-python==5.0.0.93, scipy==1.18.1, matplotlib==3.11.2
Kho mã nguồn và phiên bản	[https://github.com/thespaceio/AFIS.git]

Thông tin	Giá trị
	(https://github.com/thespaceio/AFIS.git) mã commit: 0758696
Dữ liệu	Tập FVC2002 DB4_B (vân tay tổng hợp sinh bằng phần mềm SFinGe), gồm 80 ảnh của 10 ngón tay (101_1 đến 110_8), định dạng .tif, kích thước chuẩn hóa 500x500. http://bias.csr.unibo.it/fvc2002/Downloads/DB4_B.zip
Hạt giống ngẫu nhiên	Không dùng yếu tố ngẫu nhiên
Lệnh chạy chính	1. python main.py 2. python -m scripts.batch_process 3. python -m scripts.evaluate_frr_far
Thời gian chạy	Tiền xử lý trung bình 16.82 giây/ảnh; tổng thời gian chạy kịch bản đo FRR/FAR cho 3 cấu hình là 91.68 phút

4. Các bước thực hiện và minh chứng

4.1. Bước 1: Khởi tạo Không Gian Làm Việc

Việc đã làm.

- Thiết lập thư mục Biometric_AFIS trên máy cá nhân, nhân bản mã nguồn từ kho GitHub bằng lệnh git clone. Khởi tạo môi trường ảo Python 3.13.9 cô lập, kích hoạt môi trường và cài đặt các gói phụ thuộc qua tệp requirements.txt.

Lệnh hoặc đoạn mã chính.

```
mkdir Biometric_AFIS && cd Biometric_AFIS
git clone https://github.com/thespaceio/AFIS.git .
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

Minh chứng.

```
(venv) C:\Users\DELL\Biometric_AFIS>git log -1 --oneline
0758696 (HEAD -> main, origin/main, origin/HEAD) Delete test directory

(venv) C:\Users\DELL\Biometric_AFIS>python --version
Python 3.13.9

(venv) C:\Users\DELL\Biometric_AFIS>pip list
Package            Version
-----
contourpy          1.4.0
cyclor              0.12.1
fonttools          4.66.0
kiwisolver         1.5.1
matplotlib         3.11.2
numpy              2.5.3
opencv-python     5.0.0.93
packaging          26.3
pillow             12.3.0
pip                25.2
pyparsing          3.3.3
python-dateutil    2.9.0.post0
scipy              1.18.1
six                1.17.0
```

Hình 4.1: Kết quả kiểm tra mã commit, phiên bản Python và các thư viện trong môi trường ảo

Nhận xét. Môi trường ảo Python 3.13.9 được thiết lập và kích hoạt thành công. Các thư viện phụ thuộc phục vụ xử lý ảnh và tính toán ma trận (numpy 2.5.3, opencv-python 5.0.0.93, scipy 1.18.1) đã được cài đặt đầy đủ theo yêu cầu từ requirements.txt, không phát sinh xung đột.

4.2. Bước 2: Khám phá quy trình qua giao diện dòng lệnh (CLI)

Việc đã làm.

- Sửa lỗi import module system.py trong tệp main.py. Tiến hành đăng ký mẫu ngón 101_1.tif gán định danh user01. Kiểm thử đối sánh 1:1 trong hai tình huống: cùng ngón (101_2.tif) và khác ngón (102_1.tif). Trích xuất thống kê hiệu năng bằng lệnh stats.

Lệnh hoặc đoạn mã chính.

```
python main.py
> enroll → ID: user01, Path: data/raw/101_1.tif
> verify → ID: user01, Path: data/raw/101_2.tif
> verify → ID: user01, Path: data/raw/102_1.tif
> stats
```

Minh chứng.

```

> enroll

=====
ENROLLMENT
=====
Enter person ID (e.g., 'john_doe'): user01
Select finger position (optional, press Enter to skip):
  1 - Right thumb
  2 - Right index
  3 - Right middle
  4 - Right ring
  5 - Right little
  6 - Left thumb
  7 - Left index
  8 - Left middle
  9 - Left ring
 10 - Left little
Position (1-10, or blank):
Enter image path: C:\Users\DELL\Biometric_AFIS\data\raw\101_1.tif
Processing image: C:\Users\DELL\Biometric_AFIS\data\raw\101_1.tif ...
🔴 Enrolling fingerprint for user01...
✅ Enrollment successful: 150 minutiae, pattern: arch
✅ Enrollment successful for user01.
  Template ID: user01_1_20260927104446
  Minutiae: 150
  Pattern type: arch
Show minutiae visualization? (y/n): n

> verify

=====
VERIFICATION
=====
Enter claimed person ID: user01
Enter image path: C:\Users\DELL\Biometric_AFIS\data\raw\101_2.tif
Verifying C:\Users\DELL\Biometric_AFIS\data\raw\101_2.tif against user01 ...
🔵 Verifying fingerprint for user01...
✅ Verification result: MATCH (score: 0.356)
✅ VERIFIED: The fingerprint matches user01 (score: 0.356)
  Matched minutiae: 43

```

Hình 4.2a: Kết quả thực thi quy trình đăng ký ảnh 101_1 và xác thực thành công ảnh cùng ngón 101_2 trên CLI

```

> verify

=====
VERIFICATION
=====
Enter claimed person ID: user01
Enter image path: C:\Users\DELL\Biometric_AFIS\data\raw\102_1.tif
Verifying C:\Users\DELL\Biometric_AFIS\data\raw\102_1.tif against user01 ...
🔵 Verifying fingerprint for user01...
✅ Verification result: NO MATCH (score: 0.248)
⚠️ NOT VERIFIED: Score 0.248 below threshold.
   Matched minutiae: 25

> stats

=====
SYSTEM STATISTICS
=====
Total templates: 1
Unique persons: 1
Average minutiae per template: 150.0

Pattern type distribution:
  arch: 1

Preprocessing stats:
  Images processed: 3
  Avg processing time: 16.820 s
  Avg quality score: 0.475

Feature extraction stats:
  Images processed: 3
  Avg minutiae count: 150.0

Matching stats:
  Verifications performed: 2
  Identifications performed: 0
  Average match score: 0.302

```

Hình 4.2b: Kết quả từ chối xác thực ảnh khác ngón 102_1 và bảng thống kê hiệu năng hệ thống qua lệnh `stats`

Nhận xét.

- Hệ thống phân loại dạng vân arch và trích xuất chạm trần 150 minutiae do nhiễu mô phỏng của tập DB4_B sinh nhiều điểm giả. Với ngưỡng quyết định mặc định $T = 0.35$, hệ thống chấp nhận chính xác lượt cùng ngón 101_2.tif (score: 0.356, 43 minutiae khớp) và từ chối đúng lượt khác ngón 102_1.tif (score: 0.248, 25 minutiae khớp). Lệnh `stats` ghi nhận thời gian tiền xử lý trung bình khá lâu (16.820 giây/ảnh), cho thấy chi phí tính toán trên CPU cao và cần tối ưu số lượt thử nghiệm ở các bước sau.

4.3. Bước 3: Tích hợp API vào kịch bản tự động

Việc đã làm.

- Hoàn thiện kịch bản `scripts/batch_process.py`, gọi trực tiếp `_create_template_from_image` từ `system.py` để trích xuất đặc trưng cho toàn bộ tập dữ liệu. Chuẩn hóa tọa độ x, y về đoạn $[0,1]$ bằng cách chia cho 500.0 (do hàm nạp ảnh nội bộ đã resize ảnh gốc về kích thước cố định 500×500) và trích trường hướng orientation. Lưu cấu trúc mảng đa chiều ra tệp `.npy`

Lệnh hoặc đoạn mã chính.

```
template = system._create_template_from_image(img_path, template_id)
```

```
if template and template.minutiae:
```

```
    data_list = []
```

```

for m in template.minutiae:
    # Chuẩn hóa về [0, 1] trên kích thước 500x500
    norm_x = m.x / 500.0
    norm_y = m.y / 500.0
    theta = m.orientation
    data_list.append([norm_x, norm_y, theta])
np.save(out_path, np.array(data_list))

```

Minh chứng.

```

>>> import numpy as np
>>> data = np.load('data/minutiae/101_1.npy')
>>> print("Kích thước mảng:", data.shape)
Kích thước mảng: (150, 3)
>>> print("5 dòng đầu tiên:\n", data[:5])
5 dòng đầu tiên:
[[0.304      0.032      2.81474191]
 [0.88       0.034      0.09261206]
 [0.774      0.038      3.08602041]
 [0.882      0.05       0.07855527]
 [0.488      0.062      2.47955432]]

```

Hình 4.3: Kiểm tra cấu trúc tệp 101_1.npy: kích thước mảng (150, 3) và 5 hàng tọa độ chuẩn hóa đầu tiên.

Nhận xét. Kích bản tự động hóa xử lý thành công toàn bộ tập dữ liệu ảnh, không phát sinh lỗi ngoại lệ. Dữ liệu trích xuất kiểm tra trên tệp đại diện 101_1.npy cho thấy tọa độ x, y nằm hoàn toàn trong đoạn $[0,1]$ và trường góc mang giá trị radian hợp lệ, sẵn sàng làm đầu vào cho các thuật toán so khớp đặc trưng hình học hoặc mô hình bảo vệ mẫu.

5. Kết quả định lượng

5.1. Thống kê đặc trưng Minutiae trên tập FVC2002

Công thức & Quy chuẩn: Trích xuất từ 80 ảnh của tập FVC2002 DB4_B qua kịch bản batch_process.py.

Bảng 1: Thống kê số lượng điểm minutiae trích xuất trên tập dữ liệu FVC2002

Chỉ số thống kê	Giá trị	Ngưỡng chặn hệ thống	Ghi chú
Nhỏ nhất (Min)	150	150	Chạm ngưỡng chặn trên
Trung vị (Median)	150.0	150	Phân phối hoàn toàn bão hòa
Lớn nhất (Max)	150	150	Bị giới hạn bởi tham số max_minutiae

Nhận xét về chất lượng tiền xử lý và đặc trưng:

Số lượng minutiae trích xuất trên toàn bộ các ảnh đều đạt mức cực đại 150. Hiện tượng phân phối bão hòa tuyệt đối này chứng minh rằng khâu tiền xử lý (lọc Gabor và thuật toán làm mảnh khung xương vân) tạo ra một số lượng rất lớn các minutiae giả (spurious minutiae) phát sinh từ nhiễu biên, đốm bẩn cảm biến hoặc đứt gãy vân giả tạo. Bộ lọc hậu xử lý chưa đủ chặt chẽ để loại bỏ

các điểm giả này, buộc hệ thống phải cắt ngọn ở ngưỡng trần $\text{max_minutiae} = 150$. Sự tồn tại của nhiều minutiae giả là nhân tố tiềm ẩn làm sai lệch điểm số đối sánh ở các bước tiếp theo.

5.2. Đánh giá độ lỗi FRR và FAR theo tham số tiền xử lý

- Giao thức thực nghiệm: Giao thức rút gọn (Mỗi cấu hình gồm 10 ngón tay; 30 lượt cùng ngón đo FRR, 30 lượt khác ngón đo FAR).
- Điều kiện đo: Ngưỡng quyết định cố định $T = 0.35$; tham số thay đổi: `adaptive_block_size` trong `PreprocessingPipeline`.
- Công thức:

$$\text{FRR} = \frac{\text{Số lượt cùng ngón bị từ chối}}{30} \times 100\%$$

$$\text{FAR} = \frac{\text{Số lượt khác ngón được chấp nhận}}{30} \times 100\%$$

Bảng 2: Biến thiên của tỷ lệ lỗi FRR và FAR khi thay đổi kích thước khối nhị phân hóa thích nghi

Cấu hình	Tham số thay đổi và giá trị	Cỡ mẫu	FRR (%)	FAR (%)	Số lượt từ chối do sai dạng vân (Mismatch)	Ghi chú
C0 (mặc định)	11 (Mặc định)	60 (30 Genuine / 30 Impostor)	93.33	3.33	0	Điểm khớp cao nhất chỉ xoay quanh 0.35
C1	5 (Khối nhỏ)	60 (30 Genuine / 30 Impostor)	100.00	0.00	0	Đứt gãy vân, mất đặc trưng thật
C2	21 (Khối lớn)	60 (30 Genuine / 30 Impostor)	100.00	0.00	0	Nhiều biên khối lớn làm dính vân

```
(venv) C:\Users\DELL\Biometric_AFIS>python -m scripts.evaluate_frr_far

Cấu hình          | FRR (%)      | FAR (%)      | Sai dạng vân (Pattern)
-----
[*] Đang chạy C0 (Mặc định) (block_size = 11)...
-> Đã trích xuất xong đặc trưng! Đang tính toán đối sánh...
--> C0 (Mặc định)   | FRR: 93.33% | FAR: 3.33% | Mismatch: 0 lượt

[*] Đang chạy C1 (Nhỏ) (block_size = 5)...
-> Đã trích xuất xong đặc trưng! Đang tính toán đối sánh...
--> C1 (Nhỏ)        | FRR: 100.00% | FAR: 0.00% | Mismatch: 0 lượt

[*] Đang chạy C2 (Lớn) (block_size = 21)...
-> Đã trích xuất xong đặc trưng! Đang tính toán đối sánh...
--> C2 (Lớn)        | FRR: 100.00% | FAR: 0.00% | Mismatch: 0 lượt

Tổng thời gian chạy hoàn tất: 91.68 phút
```

Hình 5.1: Minh chứng kết quả thực nghiệm đo FRR, FAR và tổng thời gian chạy 91.68 phút xuất từ chương trình `evaluate_frr_far.py`.

6. Phân tích và thảo luận

Câu 1. Phân tích hiện tượng bão hòa số lượng minutiae (Min = Median = Max = 150):

- Hiện tượng: Tất cả các ảnh sau khi qua hàm `_create_template_from_image` đều sinh ra đúng 150 điểm minutiae.
- Cơ chế: Trong module `features.py`, tham số trần `max_minutiae` được ấn định là 150. Mặc dù tập dữ liệu DB4_B là vân tay tổng hợp sinh bằng phần mềm SFinGe không có nhiều da người thật, mô hình sinh tổng hợp này lại mô phỏng các vết trầy xước nhân tạo, chấm nhiễu nền và độ biến thiên mật độ vân rất phức tạp. Thuật toán nhị phân hóa cục bộ và làm mảnh khung xương (thinning) chưa có bộ lọc thích ứng tốt, dẫn đến việc nhận diện sai các vết nhiễu mô phỏng này thành các điểm tận cùng (ending) và điểm phân nhánh (bifurcation) giả. Số minutiae phát hiện trên mỗi ảnh đều vượt quá 150, buộc hệ thống phải cắt ngọn về mức trần. Điều này chứng minh khâu hậu xử lý lọc minutiae giả của hệ thống hoạt động chưa hiệu quả, làm suy giảm tính phân biệt hình học của mẫu sinh trắc.

Câu 2. Phân tích cơ chế biến thiên của FRR và FAR khi thay đổi `adaptive_block_size`:

- Hiện tượng: Cấu hình mặc định C0 (`block_size = 11`) cho tỷ lệ từ chối sai rất cao (FRR = 93.33%). Khi chuyển sang cấu hình C1 (5) và C2 (21), hệ thống rơi vào trạng thái bế tắc toàn phần khi từ chối toàn bộ mọi lượt so khớp cùng ngón (FRR = 100.00%) và triệt tiêu hoàn toàn mạo danh (FAR = 0.00%).
- Cơ chế:
 1. **Tại C1 (`block_size = 5`):** Kích thước khối lân cận quá nhỏ khiến thuật toán tính ngưỡng cục bộ trở nên cực kỳ nhạy cảm với các đốm nhiễu hạt nền và độ biến thiên cường độ xám mô phỏng từ phần mềm SFinGe. Hậu quả là khung xương vân tay sau khi làm mảnh bị phân mảnh nghiêm trọng, các đường vân nhân tạo bị đứt gãy thành nhiều đoạn vụn. Tọa độ và trường hướng của các minutiae thật bị xáo trộn, đồng thời làm bùng nổ hàng loạt minutiae giả dạng tận cùng (ending). Điều này khiến cấu trúc không gian giữa hai lần sinh mẫu của cùng một ngón không thể tìm được tập điểm tương đồng, điểm đối sánh luôn nằm dưới ngưỡng 0.35, dẫn đến $FRR = 100.00\%$.

2. **Tại C2 (block_size = 21):** Kích thước khối lân cận quá rộng dẫn đến hiện tượng trôi ngưỡng tại các khu vực chuyển tiếp giữa các đường vân lân cận có tần số vân dày đặc. Thuật toán nhị phân hóa làm mất chi tiết vi mô, khiến các rãnh vân (valleys) bị phủ đen và dính bết các đường vân lồi vào nhau. Kết quả là các minutiae trích xuất được chủ yếu là các điểm rẽ nhánh giả xuất hiện tại rìa các khối bết dính. Giữa các lần sinh mẫu khác nhau của cùng một ngón, do phần mềm SFinGe áp dụng các phép biến dạng phi tuyến và xoay/tịnh tiến mô phỏng khác nhau, vị trí các khối dính này không trùng khớp, tạo ra hai tập minutiae giả hoàn toàn sai lệch và đầy điểm so khớp rơi về dưới 0.35 ($FRR = 100.00\%$).
3. Lý do từ chối sai dạng vân: Cột Mismatch ghi nhận 0 lượt cho thấy thuật toán phân loại dạng vân tổng thể (arch, loop, whorl) hoạt động ổn định; sự gia tăng đột biến của FRR hoàn toàn do khâu trích xuất tọa độ minutiae bị hỏng từ bước tiền xử lý.

Giới hạn kết luận: Quy mô khảo sát 60 lượt đối sánh trên tập con 10 ngón của DB4_B; độ phân giải cố định ở 500 dpi. Theo quy tắc ba (rule of three), với cỡ mẫu $n=30$ lượt khác ngón, giá trị $FAR = 0.00\%$ có cận trên độ tin cậy 95% là xấp xỉ $3/30 \approx 10\%$.

7. Ý nghĩa đối với bảo mật

Kết quả thực nghiệm minh chứng rõ nét quy luật đánh đổi căn bản trong an toàn sinh trắc học: tối ưu hóa sự tiện dụng (Usability) luôn xung đột trực tiếp với tính bất khả xâm phạm (Security).

Việc cấu hình C1 và C2 đưa FAR về 0.00% trên lý thuyết tạo ra một hệ thống phòng thủ tuyệt đối trước các đòn tấn công mạo danh (Zero-Effort Impostor Attack), nhưng lại phá hủy hoàn toàn khả năng vận hành thực tế khi biến hệ thống thành một cuộc tấn công từ chối dịch vụ tự thân (Denial of Service) với người dùng hợp lệ ($FRR = 100\%$). Cấu hình C0 tuy kiểm soát được FAR ở mức 3.33%, nhưng $FRR = 93.33\%$ vẫn là rào cản quá lớn để ứng dụng dân sự.

Nếu bắt buộc phải triển khai trong môi trường thương mại thực tế:

1. Em chọn giữ cấu hình tiền xử lý gốc C0 (adaptive_block_size = 11) nhưng cần hạ ngưỡng quyết định min_matches_ratio từ 0.35 xuống khoảng 0.25 - 0.28 để đưa FRR về vùng chấp nhận được (10% - 15%), đồng thời bù đắp lỗ hổng FAR tăng bằng mô-đun phát hiện thực thể sống (Liveness Detection).
2. Về mặt lưu trữ an toàn, cơ sở dữ liệu mẫu sinh trắc (*.db) trong bài thực hành đang lưu tọa độ minutiae dạng thô, rất dễ bị kẻ tấn công dịch ngược tái tạo lại ảnh vân tay ban đầu. Khi triển khai thương mại bắt buộc phải áp dụng kiến trúc Sinh trắc học có thể hủy (Cancelable Biometrics) hoặc Hàm sinh trắc (Biometric Cryptosystem) để biến đổi các vector tọa độ một chiều không thể đảo ngược.

8. Sự cố gặp phải và cách xử lý

Sự cố (thông báo lỗi)	Nguyên nhân	Cách xử lý
ModuleNotFoundError: No module named 'afis_core' khi chạy main.py	Kho mã nguồn không có tệp afis_core.py; tác giả đã tái cấu trúc và đổi tên module chứa lớp AFISSystem thành system.py nhưng chưa cập nhật import trong main.py	Sửa dòng 14 trong main.py từ "from afis_core import AFISSystem" thành "from system import AFISSystem"
TypeError: AFISSystem._create_templa	Phương thức _create_template_from_image	Cập nhật kịch bản batch gọi hàm với đủ 2 tham số:

Sự cố (thông báo lỗi)	Nguyên nhân	Cách xử lý
te_from_image() missing 1 required positional argument: 'template_id'	yêu cầu bắt buộc phải truyền đủ 2 tham số là đường dẫn ảnh và định danh mẫu.	system._create_template_from_image(img_path, template_id)
TypeError: '<' not supported between instances of 'tuple' and 'float' tại dòng so sánh điểm đối sánh	Hàm “match_templates” trong module matching.py trả về một tuple gồm điểm số và cấu trúc phụ (thay vì giá trị float đơn thuần).	Viết hàm hỗ trợ “parse_score()” để bóc tách phần tử điểm số đầu tiên result[0] trước khi so sánh với ngưỡng THRESHOLD

9. Dữ liệu sinh trắc, nguồn tham khảo và công cụ AI

Dữ liệu sinh trắc. Đánh dấu vào các ô sau:

- ☒ Bài nộp không chứa ảnh vân tay, khuôn mặt, móng mắt hay giọng nói của người thật, trừ khi hướng dẫn của bài yêu cầu.
- ☒ Bài nộp không chứa cơ sở dữ liệu mẫu (*.db, *.pk1), embedding, trọng số mô hình, thư mục môi trường ảo hay tập dữ liệu tải về.

Nguồn tham khảo.

- Kho mã nguồn hệ thống nhận dạng vân tay AFIS: [https://github.com/thespaceio/AFIS.git] (https://github.com/thespaceio/AFIS.git) (Truy cập ngày 27/09/2026).
- Cơ sở dữ liệu chuẩn FVC2002 (Fingerprint Verification Competition 2002).

Công cụ AI.

- Sử dụng mô hình Gemini để hỗ trợ phân tích thông báo lỗi cú pháp Python (ModuleNotFoundError, TypeError), gợi ý cấu trúc kịch bản tối ưu tiến trình đo đạc và định dạng số liệu theo mẫu học thuật. Toàn bộ các dòng lệnh, kịch bản kiểm thử và các con số thực nghiệm (91.68 phút, FRR, FAR) do sinh viên tự chạy thực tế và chịu trách nhiệm giải trình.

10. Cam kết

Tôi cam kết các kết quả trong báo cáo này do chính tôi chạy trên máy của mình, các phần sử dụng lại của người khác đã được ghi nguồn đầy đủ.

Nguyễn Huy Đạt, 27/9/2026

Phụ lục A. Mã nguồn chính

1. Sửa lỗi import trong main.py (dòng 14)

```
# Sửa tên module bị thiếu afis_core thành system
from system import AFISSystem
```

2. Đoạn trích xuất và chuẩn hóa tọa độ (scripts/batch_process.py)

```
# Gọi trích xuất đặc trưng với đầy đủ 2 tham số
template = system._create_template_from_image(img_path, template_id)
if template and template.minutiae:
    # Chuẩn hóa x, y về [0, 1] do ảnh gốc đã bị resize về 500x500; lấy góc orientation
```

```
data = np.array([[m.x / 500.0, m.y / 500.0, m.orientation] for m in template.minutiae])
np.save(out_path, data)
```

3. Hàm bóc tách điểm và cập nhật cấu hình đo lỗi (scripts/evaluate_frr_far.py)

```
def parse_score(result):
    """Trích xuất giá trị số thực từ tuple trả về của match_templates để tránh lỗi so sánh."""
    return float(result[0]) if isinstance(result, tuple) else float(result.get('score', 0.0)) if
    isinstance(result, dict) else result
```

```
# Cập nhật tham số nhậ phân hóa ngay lúc khởi tạo PreprocessingPipeline
system.preprocessor = PreprocessingPipeline(config={"adaptive_block_size":
cfg["adaptive_block_size"]})
```

Phụ lục B. Nhật ký chạy

```
>>> import numpy as np
>>> data = np.load('data/minutiae/101_1.npy')
>>> print("Kích thước mảng:", data.shape)
Kích thước mảng: (150, 3)
>>> print("5 dòng đầu tiên:\n", data[:5])
5 dòng đầu tiên:
[[0.304  0.032  2.81474191]
 [0.88   0.034  0.09261206]
 [0.774  0.038  3.08602041]
 [0.882  0.05   0.07855527]
 [0.488  0.062  2.47955432]]
>>> exit()
```

(venv) C:\Users\DELL\Biometric_AFIS>python -m scripts.evaluate_frr_far

Cấu hình | FRR (%) | FAR (%) | Sai dạng vân (Pattern)

[*] Đang chạy C0 (Mặc định) (block_size = 11)...

-> Đã trích xuất xong đặc trưng! Đang tính toán đối sánh...

--> C0 (Mặc định) | FRR: 93.33% | FAR: 3.33% | Mismatch: 0 lượt

[*] Đang chạy C1 (Nhỏ) (block_size = 5)...

-> Đã trích xuất xong đặc trưng! Đang tính toán đối sánh...

--> C1 (Nhỏ) | FRR: 100.00% | FAR: 0.00% | Mismatch: 0 lượt

[*] Đang chạy C2 (Lớn) (block_size = 21)...

-> Đã trích xuất xong đặc trưng! Đang tính toán đối sánh...

--> C2 (Lớn) | FRR: 100.00% | FAR: 0.00% | Mismatch: 0 lượt

Tổng thời gian chạy hoàn tất: 91.68 phút